

Санкт-Петербургский государственный университет

Искусственный интеллект и наука о данных

Реализация и сравнение алгоритмов проверки изоморфизма гиперграфов отношений на Python и C++

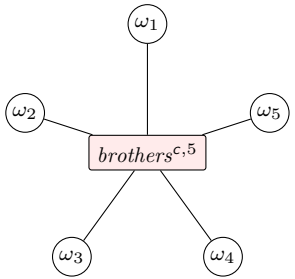
Сидоров Вячеслав Павлович, группа 23.Б16-мм

Научный руководитель: д.ф.-м.н., профессор Т.М. Косовская

Санкт-Петербург
2026

Введение

- Сложные структурированные объекты описывают их элементами, свойствами и отношениями — формулами исчисления предикатов
- В формуле у отношения фиксированы арность и порядок аргументов: «пятеро — братья» — 20 атомарных формул
- *Гиперграфы отношений* Т. М. Косовской: коммутативное отношение любой арности — одно гиперребро
- Для них предложены алгоритмы проверки изоморфизма ISOM-1 и ISOM-n; программной реализации и экспериментальной оценки пока нет



одно ребро типа (2) вместо 20 формул «x_i — брат x_j»

Гиперграфы отношений и их изоморфизм

- Рёбра: тип (1) $p^k \to (\omega_{i_1}, \ldots, \omega_{i_k})$ — кортеж, тип (2) $p^{c,k} \to \{\omega_{i_1}, \ldots, \omega_{i_k}\}$ — множество

Определение

G_1 и G_2 изоморфны, если есть гиперграф H с переменными вместо имён объектных вершин и подстановки (унификаторы) λ_1, λ_2: $H\lambda_1 = G_1$, $H\lambda_2 = G_2$

- H получают из G_2 заменой констант переменными и ищут биекцию λ с $H\lambda = G_1$
- *Характеристика* вершины: вектор $(n_1, \ldots, n_k)$ вхождений по местам (тип 1) или число вхождений (тип 2); вершины с равной характеристикой образуют классы
- *Структурное отображение* сопоставляет классы переменных и констант: перебор $\prod_k t_k!$ вместо $n!$
  например, p^3 на 6 вершинах: $\{[x_5] : [a_4],\ [x_3] : [a_3],\ [x_2, x_6] : [a_1, a_5],\ [x_1, x_4] : [a_2, a_6]\}$ — 4 подстановки вместо 720

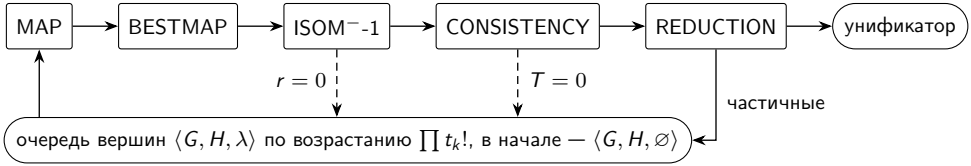
Постановка задачи

Целью работы является реализация на Python и C++ алгоритмов проверки изоморфизма гиперграфов отношений из статьи Т. М. Косовской и экспериментальное сравнение этих реализаций.

Задачи:

1. Разобрать алгоритмы ISOM-1 и ISOM-n
2. Реализовать алгоритмы на Python
3. Реализовать алгоритмы на C++ так, чтобы реализация выполняла тот же поиск, что и Python-версия
4. Разработать наборы тестовых задач и средства автоматического сравнения реализаций
5. Проверить корректность реализаций и сравнить их время работы

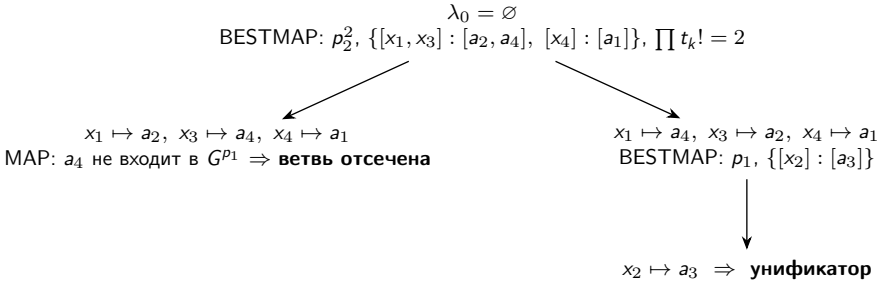
Алгоритм ISOM-n



- MAP: структурные отображения для подграфов G^i, $H^i\lambda$, где остались переменные
- BESTMAP: подграф i_0 с наименьшим числом возможных подстановок $\prod t_k!$
- ISOM$^-$-1: локальные унификаторы G^{i_0} и $H^{i_0}\lambda$ (ISOM-1 без шагов 1–3)
- CONSISTENCY: только те, что вместе с λ образуют биекцию
- REDUCTION: подграфы без переменных сравниваются с G; если переменных не осталось — унификатор

Пример работы ISOM-n

$$G_1 = \{p_1 \to a_1,\ p_1 \to a_2,\ p_1 \to a_3,\ p_2^2 \to (a_1, a_2),\ p_2^2 \to (a_1, a_4),\ p_3^{c,3} \to \{a_2, a_3, a_4\}\}$$
$$G_2 = \{p_1 \to b_2,\ p_1 \to b_3,\ p_1 \to b_4,\ p_2^2 \to (b_4, b_3),\ p_2^2 \to (b_4, b_1),\ p_3^{c,3} \to \{b_1, b_2, b_3\}\},\quad H\colon b_i \to x_i$$



Вместо $4! = 24$ подстановок рассмотрено три

Реализация

Python

- Части названы как в статье: `build_mapping`, `isom_minus_1`, `map_procedure`, `bestmap`, `consistency`, `reduction`, `isom_n`
- Подстановка — словарь, рёбра — `Counter`
- Пошаговый журнал поиска, самопроверка полным перебором

C++17

- Вершины — целые числа, подстановка — массив, рёбра G^i — хеш-таблица
- Тот же порядок обхода: `std::next_permutation` $\leftrightarrow$ `itertools.permutations`, куча с ключом ($\prod t_k!$, номер)
- Тот же журнал поиска, что и у Python-версии



Корректность

Лемма

Если каждое ребро $H^i\lambda$ проверять один раз — после класса, на котором в нём не остаётся переменных, — ISOM$^-$-1 отсекает те же подстановки, что и при проверке всех рёбер после каждого класса.

Лемма

Если BESTMAP выбирает подграф однозначно, все вершины дерева ISOM-n и все найденные унификаторы различны: «склейка» вершин не нужна.

Проверка реализаций

- Полный перебор всех $n!$ биекций на малых задачах (обе реализации, $n \leq 8$)
- Сверка Python и C++: ответы, множества изоморфизмов, статистика деревьев, журналы
- 454 задач, 3095 проверок — **расхождений нет**

Экспериментальное исследование

Стенд: Intel Core i5-10210U, частота зафиксирована на 1,6 ГГц, процесс на одном ядре; CPython 3.14, g++ 15.2 -O2

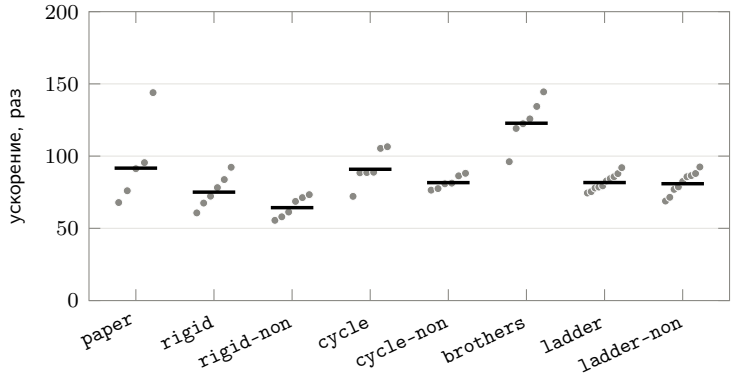
Методика:

- прогрев и 40 замеров на задачу
- задачи короче 20 мс — пачками, как в `timeit`
- среднее $\pm$ с.о.; с.о. $< 10\,\%$ во всех 112 сериях

Набор	Что проверяет	Размер
`paper`	примеры из статьи	$n \leq 6$
`random`	300 случайных пар, сверка с перебором	$n \leq 7$
`random-mid`	100 случайных пар, 3–5 предикатов	$n \leq 20$
`rigid(-non)`	классы из одной вершины, без ветвлений	$n \leq 768$
`cycle(-non)`	один класс: перебор $n!$	$n \leq 9$
`brothers`	одно ребро: $k!$ унификаторов	$k \leq 8$
`ladder(-non)`	дерево поиска из $2^d - 1$ вершин	$d \leq 11$

Варианты `-non` — неизоморфные пары с совпадающими характеристиками

Результаты: ускорение C++ относительно Python



точки — задачи, черта — среднее геометрическое по набору

84×

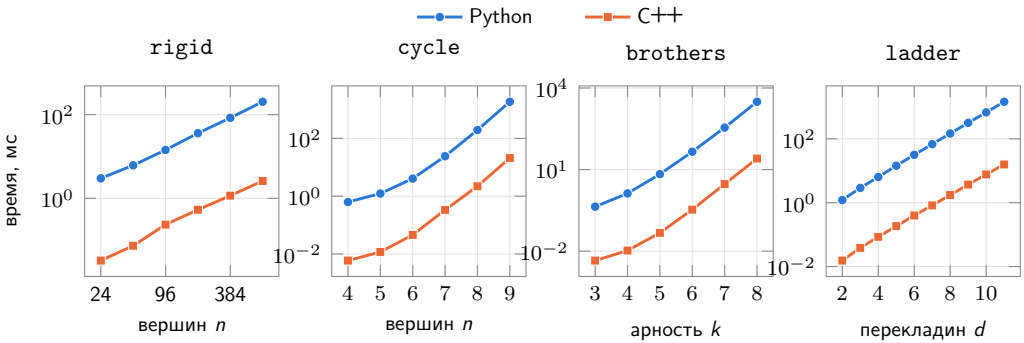
среднее геометрическое

56–144×

по всем задачам

Обе реализации делают одни и те же шаги, поэтому разница почти постоянна; в Python время уходит на операции интерпретатора

Результаты: зависимость от размера задачи



- Кривые параллельны: реализации различаются постоянным множителем, характер роста задаёт алгоритм
- `rigid` — почти линейный рост; `cycle`, `brothers` — как $n!$, $k!$; `ladder` — как 2^d, в согласии с числом возможных подстановок $\prod t_k!$

Результаты

- Разобраны алгоритмы ISOM-1 и ISOM-n; показано, что при однозначном выборе подграфа все вершины дерева поиска различны
- Алгоритмы реализованы на Python с пошаговым журналом поиска и самопроверкой полным перебором
- Алгоритмы реализованы на C++17 с тем же порядком обхода дерева поиска
- Разработан сценарий сравнения: 10 наборов задач, сверка ответов, изоморфизмов, деревьев поиска и полного перебора, замеры по методике с контролем разброса
- Реализации проверены на 454 задачах (3095 проверок, расхождений нет); C++ быстрее в 56–144 раз, в среднем в 84 раз

Код и отчёт: `https://github.com/xamelllion/isom`

Дополнительно: время на самых больших задачах

Задача	n	$\lvert E\rvert$	Вершин дерева	Python, мс	C++, мс	Ускорение
`rigid_n768`	768	4860	1	203 ± 8	$2{,}59 \pm 0{,}02$	78×
`rigid_non_n768`	768	4860	1	151 ± 7	$2{,}202 \pm 0{,}013$	69×
`cycle_n9`	9	9	1	1856 ± 12	$21{,}0 \pm 0{,}9$	88×
`cycle_non_n9`	9	9	1	1840 ± 15	$21{,}3 \pm 0{,}3$	86×
`brothers_k8`	8	1	1	3110 ± 50	$25{,}4 \pm 0{,}2$	122×
`ladder_d11`	22	33	2047	1447 ± 15	$15{,}73 \pm 0{,}18$	92×
`ladder_non_d11`	22	33	2047	1453 ± 20	$15{,}7 \pm 0{,}4$	92×
`paper`: 5 задач	4–6			$9{,}63 \pm 0{,}12$	$0{,}082 \pm 0{,}003$	118×
`random`: 300 задач	1–7			158 ± 12	$2{,}52 \pm 0{,}12$	63×
`random-mid`: 100 задач	8–20			430 ± 20	$7{,}2 \pm 0{,}2$	60×

Среднее $\pm$ стандартное отклонение по 40 замерам; для `paper` — сумма по задачам, для `random` и `random-mid` — все задачи набора подряд

Дополнительно: как обеспечен одинаковый поиск

Что упорядочено	Python	C++
перестановки класса	`itertools.permutations` от-сортированного списка	`std::next_permutation` от возрастающего массива
очередь	`heapq`, ключ ($\prod t_k!$, счётчик)	`std::push_heap`, тот же ключ
предикаты в BESTMAP	по имени предиката	по той же строке
классы отображения	по характеристике, затем по имени	по характеристике, затем по номеру

Совпадение подтверждается статистикой поиска (вызовы MAP, вершины дерева, кандидаты r и T, полные и частичные унификаторы) и построчным совпадением журналов